Design and Implementation of a Responsive E-Commerce Storefront Frontend for Picklymart Using HTML, CSS and Vanilla JavaScript

Author: Puja Dey

Student ID: 0222220005101191

Abstract

Online shopping has become a common way to buy electronic parts, computer hardware and office supplies. This paper describes the design and implementation of the frontend of Picklymart, an online store that sells robotics kits, electronic components, computer products and point-of-sale (POS) paper rolls. The interface was written in plain HTML5, CSS3 and JavaScript, with no framework and no build tool. It includes a sticky header, a category sidebar, a wholesale promotion banner, horizontally scrolling product rows, a searchable and sortable product grid, a shopping cart, and separate views for categories, contact, login and registration. The layout adapts to the screen through five breakpoints, from large desktop monitors down to 560 px phones. All product data is stored in a JavaScript array, so the project works as a self-contained prototype. The paper explains how the code is organised, the main design decisions, how each feature was checked, and where the current version falls short. It closes with a plan for connecting the interface to a real backend.

Keywords: e-commerce, frontend development, responsive web design, vanilla JavaScript, user interface, shopping cart

1. Introduction

Buying things online is now an ordinary habit, and for an online store the website is the shop. A visitor who cannot find a product within a few seconds, or who has to pinch and zoom to press a button on a phone, will usually leave. The part of a website that decides this experience is the frontend: the markup, styles and scripts that run inside the visitor’s browser.

Picklymart is an online store whose catalogue covers robotics kits and boards, electronic components and sensors, computer products, 3D printing supplies, drones, tools, and thermal paper rolls for POS printers. Its tagline is “Smart Picks. Smart Life.” The store serves two kinds of buyers. Hobbyists and students buy single items such as an Arduino board or a pack of jumper wires, while shops and cafés buy paper rolls in bulk. A good interface therefore has to support quick browsing for the first group and give the second group a clear route to a wholesale enquiry.

This paper reports the work done on the frontend of Picklymart. The task was to build the customer-facing interface only: the home page, category browsing, product search, a shopping cart, and the account and contact pages. No server or database is part of this work, so every product and every response is simulated inside the browser. That limit is deliberate. It allows the interface to be designed, shown and tested on its own, and it leaves a clean boundary at which a backend can be attached later.

The main contributions of the work are these. First, a home page that places the category list, a promotional banner and a deals panel side by side at the top of the page. Second, a small set of rendering functions that produce every product card on the site from one data array. Third, a cart with live totals in the header. Fourth, a layout that adjusts across five screen widths. Fifth, a record of the design decisions and the known weaknesses of the result, so that the next stage of development can start from facts instead of guesses.

The rest of the paper is organised as follows. Section 2 reviews the background ideas. Section 3 states the problem and the objectives. Section 4 lists the tools used. Sections 5 and 6 describe the design and the implementation. Section 7 covers testing, Section 8 discusses limitations, Section 9 suggests future work, and Section 10 concludes.

2. Background and Related Work

2.1 Usability in online stores

Nielsen (1993) describes usability through five qualities: learnability, efficiency, memorability, low error rates and user satisfaction. Each of them matters in a store. A first-time visitor should understand the menu without help, a returning visitor should reach a product quickly, and a mistake such as removing the wrong item from the cart should be easy to undo. Nielsen and Molich (1990) proposed heuristic evaluation, in which a designer checks an interface against a short list of principles to find problems cheaply,

before any user study is run. The same habit was followed during this project: after each feature was written, it was checked against questions such as whether the system shows what has happened and whether the wording matches what shoppers would expect.

Krug (2014) makes a related observation. People do not read web pages carefully; they scan them and click the first option that looks reasonable. For a store this means that labels must be short, prices must be easy to see, and important actions must sit where people expect them. The Picklymart interface follows this advice by keeping the cart summary in the header, giving every product card the same structure, and showing the price in bold directly below the product name.

2.2 Responsive web design

Marcotte (2010) introduced the term responsive web design for sites that combine three techniques: fluid grids, flexible images and CSS media queries. A responsive page does not need a separate mobile version. It rearranges its own content according to the width of the screen. Today this is the normal way to build a site, because the same store is visited from phones, tablets and desktop computers. The Picklymart frontend applies all three techniques, and Section 6.8 lists the breakpoints in detail.

2.3 Accessibility

The Web Content Accessibility Guidelines 2.1 (W3C, 2018) are organised around four principles: content should be perceivable, operable, understandable and robust. At the level of a student project these principles translate into concrete habits. Images need text alternatives, form fields need labels, every control must work from the keyboard, and the element that has focus must be visible. Section 6.9 explains which of these habits were applied and which gaps remain.

2.4 Frameworks and plain JavaScript

Libraries such as React or Vue help when an application has a lot of shared state and many interacting components. A store of this size, with sixteen products and one cart, does not have that problem. The browser already provides what is needed: template literals for building markup, the DOM for updating it, event delegation for handling clicks, and the native dialog element for pop-up windows (Flanagan, 2020). Choosing plain JavaScript also removes the build step, so the project runs by simply opening the HTML file, and it keeps the focus on language fundamentals rather than on a tool chain.

2.5 The native dialog element

The HTML Living Standard (WHATWG) defines the dialog element for pop-up windows. When a dialog is opened with the showModal method, the browser places it above the rest of the page, makes the content behind it inert, and lets the user close it with the Escape key (Mozilla Developer Network). These behaviours used to require extra scripts and careful focus handling. Using the native element gives the project a modal cart, search box and checkout form with very little code.

3. Problem Statement and Objectives

Picklymart needs a customer-facing interface that is clear on large screens, usable on phones, and quick to load. The work was carried out under three constraints. There is no backend, so all data must live in the browser. No framework may be used, so the code has to rely on standard web technologies. The result must open and run directly from a folder, without installation or compilation.

Within these constraints the objectives were as follows.

1. Build a home page that presents the categories, a wholesale promotion, daily deals and several product sections in a clear order.
2. Generate all product cards from a single data source so that the design stays consistent.
3. Provide a working cart with quantity controls and a running total that is visible from every part of the home page.
4. Allow visitors to find products through search, category filters and sorting.
5. Provide separate views for all categories, a single category, contact, login and registration.
6. Validate form input in the browser and give short confirmation messages.
7. Make the layout adapt to screens from large desktops to small phones.
8. Keep the code readable enough that another developer can extend it.

4. Tools and Technologies

Table 1 summarises the technologies used and the purpose of each.

Technology	Purpose in this project
HTML5	Page structure using semantic elements such as header, nav, main, aside, section, article, footer, form and dialog
CSS3	Layout with Grid and Flexbox, custom properties for colours, media queries, scroll snapping and transitions
JavaScript (ES6 and later)	Data, rendering, cart logic, search, navigation between views and form handling
Browser DOM API	Selecting elements, inserting markup, listening to events and controlling dialogs
PNG images	Fourteen product and banner images stored in the assets folder

Table 1. Technologies used in the Picklymart frontend.

The JavaScript uses arrow functions, template literals, array methods such as filter, map, reduce and find, object destructuring, and optional chaining. No external script, stylesheet or font is loaded. The only requests that leave the project are for the category images on the categories page, which are described in Section 6.1.

5. System Design

5.1 Project structure

The project consists of three code files and one folder of images. The file index.html (109 lines, 8,990 bytes) holds the static structure of the page. The file style.css (18,072 bytes) holds all the styling and is written in a compact form. The file script.js (14,342 bytes, 100 lines) holds the data and every behaviour of the site. The assets folder contains fourteen PNG images. The three code files together take about 40 KB, and the images take about 618 KB.

Keeping one file for each concern follows the usual separation of structure, presentation and behaviour. The HTML contains only the parts that never change, such as the header, the empty containers for product rows, the footer and the dialog windows. Everything that depends on data is inserted later by JavaScript.

5.2 Home page layout

The home page is arranged from top to bottom as follows.

The header is fixed to the top of the screen while the visitor scrolls. It is 110 px high on desktop and contains the Picklymart logo, three navigation buttons (Category, Contact Us and Wholesale), a Join as Seller button, a search icon, a cart summary showing the total price and the number of items, and the Login and Registration links.

Below the header is a three-column area. On the left is a sidebar listing the eleven product categories and taking 18 percent of the width. In the centre is a large banner announcing bulk pricing for thermal and POS paper rolls, with a button labelled “Get a Wholesale Quote” and a strip at the bottom listing roll widths, core sizes and custom sizes. On the right, taking 15.2 percent of the width, is a deals panel with four products.

After this area come four product sections: Featured Products, Best Selling, Robotics Kits and Boards, and Electronic Components and Sensors. The first two are bordered boxes with arrow buttons that scroll a row of products sideways. The last two are placed on a light grey background beside a round illustration and include a View All link. The page ends with the All Products section, which has a search box, filter buttons, a sorting menu and a grid of cards, followed by the footer.

5.3 Visual design

The design uses a restrained palette so that product photos stay the centre of attention. A dark green (#1d4b40) is used for the header, the deals panel and the main buttons. A lighter green (#8fbd7c) is used as an accent on the logo and the banner button. Cards are white with thin grey borders (#e2e5e8). Blue (#2585ee) is used only for View All links. All of these colours are defined once as custom properties on the root element, which means the theme can be changed by editing a few lines. The text is set in Arial or Helvetica, which are installed on nearly every device, so no font file has to be downloaded.

5.4 Information architecture

The store defines eleven categories. Sixteen sample products are distributed among them, as Table 2 shows.

Category	Number of products
Robotics Kits and Boards	6
Electronic Components and Sensors	4

Category	Number of products
Pos Roll	3
Robotics Accessories	1
Computer	1
Thermal Sticker	1
3D Printing; Drones and Drone Accessories; Tools and Lab Equipment; Rubik's Cubes and Speedcubing; Home Decor	0 each

Table 2. Distribution of the sample products across categories.

The five empty categories are kept on purpose. They show the full structure of the store and make it possible to test how the interface behaves when a category has no products. The prices of the sample products range from ৳1.00 to ৳5,500.00 and are placeholder values for demonstration, not real prices.

6. Implementation

6.1 Data layer

Every product is an object with an identifier, a name, a numeric price, a category and an image path. The products are stored in one array at the top of script.js. Prices are kept as numbers and are turned into text only at the moment they are displayed. A small helper function named money adds the taka sign and formats the value with thousands separators and two decimal places, so 5500 is shown as ৳5,500.00. Keeping prices numeric avoids a common class of errors in which text values are added together or sorted alphabetically instead of numerically.

A second array holds the data for the categories page. Each entry has the category name, a URL fragment and an image address. These images are loaded from the Picklymart server. Because a remote image can fail to load, each one has an error handler that swaps in a local image and then removes the handler itself, so that a failing fallback cannot trigger an endless loop.

6.2 Rendering product cards

The function productCard receives a product and a flag that says whether the card should include an Add to cart button. It returns the markup for a single article element containing the image, the name and the price. Images are loaded lazily, so pictures far down the page are fetched only when the visitor scrolls near them. A second function, renderTrack, takes a container identifier and a list of products and fills the container with cards.

The four product rows on the home page are filled by choosing items from the main array by position, and the All Products grid and the category pages use the same card function. As a result, a change to the design of a card needs to be made in only one place. One caution applies. The markup is inserted through the innerHTML property, which is safe only while the data comes from the developer. Once product names come from a database or from sellers, they must be escaped first. This point is raised again in Section 8.

6.3 Shopping cart

The cart is a plain object that maps a product identifier to a quantity. It starts with one unit of the first product, which is why the header shows ৳30.00 and one item when the page is first opened. The function updateCart is the single place where the cart is displayed. It reads the entries with a quantity above zero, adds up the quantities to get the item count, multiplies each quantity by the price of the matching product to get the total, and then writes the results into three places: the header total, the header item count and the total inside the cart window. It also rebuilds the list of cart rows.

Each row has a minus and a plus button. A click changes the quantity by one, and when the quantity reaches zero the entry is removed. When no entries remain, the window shows the message “Your cart is empty.” Because every cart action ends with a call to updateCart, the header and the cart window cannot show different numbers.

The Proceed to Checkout button opens a checkout form that asks for the full name, delivery address and phone number. If the cart is empty, a message appears instead of the form. On submission the form is cleared and a confirmation message is shown. In this prototype no order is created, since there is no server to receive it.

6.4 Search, filtering and sorting

The function renderAll builds the list of products for the All Products grid. A product is included when two conditions are true. Its category must match the active filter, or the filter must be set to All. And its name and category, written together in lower case, must

contain the search text, also in lower case. Case differences therefore never prevent a match, and a search for the word robotics finds products through their category name as well as their title.

Sorting is applied to the filtered list. For the two price options the function subtracts one price from the other, and for the name option it uses localeCompare, which handles alphabetical order correctly. The default option keeps the original order of the array. If the final list is empty, the message “No products found.” is shown.

There are two ways to start a search: the box next to the All Products heading and a pop-up opened from the search icon in the header. Both write the text into the same variable. The pop-up also resets the category filter to All, because a visitor who searches from the header does not expect an earlier filter to hide the results. After a search the page scrolls smoothly to the product grid, and the button of the active filter is highlighted so that the current state is always visible.

6.5 Moving between views

Instead of creating a separate HTML file for each page, the site uses one document and swaps its content. The function showPage hides the main section and the footer, reveals an empty container, fills it with new markup and scrolls to the top. The function showHome reverses these steps. Using this approach, four additional views are built from template strings: the list of all categories, the products of one category, the contact page, and the login and registration page. The last one is a single function that receives a flag and adjusts the title, the fields and the button text, so it avoids duplicating a large block of markup.

The category views include a breadcrumb trail with buttons that lead back to the home page or to the list of categories. When a visitor opens a category, the products are selected by comparing the category name exactly, and if none are found the message “No products found in this category yet.” is displayed.

6.6 Dialogs, forms and messages

The page defines five native dialog elements: search, cart, a shared information form, checkout, and an account dialog that has since been replaced by the full login and registration pages. The function openDialog calls showModal on the chosen element. A close button inside each dialog carries a data attribute with the identifier of its dialog, and a click on the darkened area outside the window also closes it, which is detected by

checking whether the click target is the dialog itself. The Wholesale and Join as Seller buttons reuse the same information dialog and only change its title and its introductory sentence.

Forms rely on the validation built into HTML. The attributes required, type email, type tel and minlength on the password fields let the browser reject bad input and display its own message before any script runs. JavaScript then only needs to prevent the default page reload and show a confirmation. The registration form adds one check of its own: it compares the two password fields and shows the message “Passwords do not match” if they differ. No password is stored or sent anywhere.

Confirmation messages are shown by a small notification element called a toast, which appears at the bottom right of the screen for 2.4 seconds. The same element is reused each time. Any pending timer is cleared before a new one is set, so two actions in quick succession do not cause the second message to disappear early. The element has the status role so that screen readers announce it.

6.7 Event handling

Most clicks are handled by a single listener attached to the document. When a click occurs, the code uses the closest method to look for the nearest element with a relevant data attribute, such as one for adding to the cart, changing a quantity, scrolling a row, closing a dialog or running a named action. This pattern is known as event delegation. Its main advantage here is that buttons created later by JavaScript, such as the Add to cart buttons inside a freshly rendered grid, work without any extra setup. It also means the page needs one listener instead of dozens.

Static elements that exist from the beginning, such as the sidebar buttons and the filter buttons, receive their own listeners. Forms and category cards that are created only when a view is opened are handled by additional delegated listeners on the document, since they do not exist when the script first runs.

6.8 Responsive layout

The layout relies on CSS Grid for the page areas and the product grids, and on Flexbox for the header, the filter row and the cart rows. The horizontally scrolling product rows are grids that flow in columns, with overflow set to scroll on the horizontal axis, scroll snapping so a card always stops cleanly at the edge, and the scrollbar hidden for a tidy

look. The arrow buttons move the row by 80 percent of its visible width. Images use a fixed height with the object-fit property set to contain, so photos of different proportions are never stretched.

Five breakpoints control how the page changes. They are listed in Table 3.

Maximum width	Main changes
1400 px	Header spacing is tightened, the sidebar grows to 19 percent and the deals panel to 17 percent, and the banner height is reduced from 620 px to 540 px
1000 px	The header wraps and the navigation moves to its own row, the deals panel is hidden, the sidebar becomes a fixed 190 px column, and the product grid changes from five to four columns
900 px	The categories page changes from four to three columns, and the contact and account pages switch to a single column
650 px	The header becomes compact, the sidebar becomes a horizontally scrolling strip above the banner, the banner image and the round illustrations are hidden, and the product grid shows two columns
560 px	The categories page shows two columns and the contact form and its button fill the full width

Table 3. Breakpoints and the layout changes made at each one.

The viewport meta tag in the HTML head tells mobile browsers to use the real device width, and without it none of the media queries would take effect on a phone.

6.9 Accessibility measures

Several accessibility practices were applied. The document declares its language. Every product image has an alternative text that repeats the product name. Buttons that contain only an icon, such as the search, cart, login and scroll arrows, carry an aria-label. The main navigation and the category sidebar are labelled as landmarks. A visible outline

is drawn around any button, link or field that receives keyboard focus. Every form field sits inside a label element, so clicking the text focuses the field. The native dialog elements supply focus handling and Escape-key closing without extra code.

Some gaps remain. The icons in the category sidebar and the header are Unicode characters, and the same globe symbol is used for several categories, so the icons add little meaning. A full audit with an accessibility testing tool has not been done. These points are listed in Section 8.

7. Testing and Results

7.1 Method

The interface was checked by functional scenarios in a desktop browser, with the window resized to the widths of the breakpoints in Table 3. Each scenario names an action and the behaviour the code is written to produce. Table 4 lists the main ones.

No.	Scenario	Expected result
1	Click a product in the deals panel	The item count in the header increases by one, the total is updated and the message “Added to cart” appears
2	Press the plus and minus buttons in the cart	The quantity and the total change together, and a row disappears when its quantity reaches zero
3	Search for the word arduino	Only the Arduino UNO Board and the Arduino Mega 2560 Board are shown
4	Search for text that matches nothing	The message “No products found.” appears
5	Sort by price from low to high	The cheapest product (৳1.00) comes first and the most expensive (৳5,500.00) comes last
6	Select the Pos Roll filter	Exactly three products are shown
7	Open a category that has no products	The message “No products found in this category yet.” appears

No.	Scenario	Expected result
8	Submit the registration form with two different passwords	The message “Passwords do not match” appears and the form is not cleared
9	Remove all items and press Proceed to Checkout	The message “Your cart is empty” appears and the checkout form does not open
10	Close a dialog with the close button, the Escape key and a click outside it	The dialog closes in all three cases
11	Reduce the window to 600 px	The product grid shows two columns and the sidebar becomes a horizontal strip

Table 4. Functional scenarios and expected results.

7.2 Observations

The whole project, including all images, comes to 673,821 bytes, which is about 658 KB. Of this, roughly 40 KB is code and 618 KB is images. Two images, the relay module and the robotics banner, account for about 41 percent of the image weight, so compressing just those two files would give the largest saving. The page needs no external library, stylesheet or font. Apart from the optional category images on the categories page, it makes no network request, which means it loads quickly and also works offline once the files are on the computer.

The structure of the code supports these results. Because the cart display passes through one function and all cards are built by one function, changes made in one place appear everywhere, and the number of places where a bug can hide is small.

8. Discussion and Limitations

The project meets its main objectives. It provides a complete browsing path, from the home page to a category, a search, the cart and the checkout form, and it adapts to different screen sizes using only standard web technologies. The code is short enough to be read in one sitting, which supports the goal of easy extension.

The current version nevertheless has limits that should be stated plainly.

1. **Static data.** Products live inside script.js, so adding or editing one means changing code.

2. **No persistence.** The cart is held in memory. Reloading the page resets it to the starting state. The checkout form does not create an order and does not empty the cart.
3. **No server communication.** The login, registration, contact, wholesale and seller forms only show a confirmation. Nothing is authenticated or stored.
4. **No browser history.** Views are switched by JavaScript without the History API, so the Back button leaves the site instead of returning to the previous view, and individual views cannot be bookmarked or shared by link.
5. **Empty categories.** Five of the eleven categories have no products yet.
6. **Placeholder images.** Some images are reused for different products. For example, the same paper roll picture is used for two roll sizes, and one board image is used for both the Arduino Mega and the LEGO robotics kit.
7. **Unescaped markup.** As noted in Section 6.2, product text is inserted with innerHTML. This is safe for fixed data but not for user-supplied data.
8. **Accessibility not audited.** The practices in Section 6.9 were applied, but no formal audit was run, and colour contrast has not been measured.
9. **Manual testing only.** There are no automated tests, so a later change could break a feature without any warning.
10. **Image weight.** The images are not optimised, and a few large files make up much of the page size.

9. Future Work

The most important next step is to connect the frontend to a backend. A REST API could supply products, categories and orders in JSON format, replacing the arrays in script.js. The rendering functions would not need to change much, because they already accept plain product objects. At the same time, the text inserted into the page should be escaped or built with safe DOM methods.

User accounts should be implemented with proper authentication, and the cart should be saved, either in the browser's local storage for visitors who are not logged in or on the server for those who are. Payment is the next major feature. For Bangladeshi customers, mobile payment services such as bKash and Nagad and card gateways such as SSLCommerz would be natural choices, and cash on delivery is also commonly offered.

Several improvements to the interface are also planned. A product detail page with several photos and a description would help buyers decide. Routing based on the History API would give each view its own address and make the Back button work. Pagination or

infinite scrolling would be needed once the catalogue grows beyond a few dozen items. Images should be converted to modern formats such as WebP and supplied in several sizes. Finally, an automated accessibility check with a tool such as Lighthouse or axe, together with automated browser tests, would catch problems earlier than manual checking can.

10. Conclusion

This paper described the frontend of Picklymart, an online store for robotics, electronics, computer products and POS paper rolls. The interface was built with HTML, CSS and JavaScript alone and includes a sticky header, a category sidebar, a wholesale banner, scrolling product rows, search, filtering, sorting, a cart, a checkout form, and separate category, contact, login and registration views. A single data array and a small set of rendering functions keep the design consistent, and five breakpoints let the page adjust from desktop monitors to small phones.

The work shows that a store interface of this size can be built clearly and compactly without a framework, provided that the code is organised around a few well-defined functions. It also has real limits. The data is fixed, the cart is not saved, the forms do not reach a server and the views have no addresses of their own. These limits are not flaws in the design so much as the natural boundary of a frontend-only prototype, and they define the work that comes next: a backend, authentication, persistence, payment and automated testing. With those in place, the present interface can serve as the customer-facing part of a complete online store.

References

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.

Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability* (3rd ed.). New Riders.

Marcotte, E. (2010, May 25). Responsive Web Design. *A List Apart*.

Mozilla Developer Network. (n.d.). The HTML Dialog element. *MDN Web Docs*.

Nielsen, J. (1993). *Usability Engineering*. Academic Press.

Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)* (pp. 249–256). ACM.

W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation, 5 June 2018.

WHATWG. (n.d.). *HTML Living Standard*. Web Hypertext Application Technology Working Group.